

Java Syntax

Dateiname "HelloWorld.java" MUSS mit dem Klassennamen übereinstimmen.

```
package de.ostfalia.cheatsheet
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello World");
    }
}
```

Operatoren

Rang von Operatoren

Klammern haben die höchste Priorität.

Rang	Operator	INFO
1	++, --	Inkrement/Dekrement
1	+, -	Unäres Plus/Minus
1	~	invertiert alle Bits
1	!	!true = false / !false = true
1	(typ)	Casting
2	*, /, %	Mal, Geteilt, Rest
3	+, -	Addition, Subtraktion
3	+	String-Konkatenation
4	<<	Verschiebung links
4	>>	Rechtsshifting mit Vorzeichenerweiterung
4	>>>	Rechtsshifting ohne Vorzeichenerweiterung
5	<, <=, >, >=	Vergleich
5	instanceof	Typvergleich
6	==, !=	Wert
6	==, !=	Referenzen
7	&	Bitweises UND
7	&&	Logisches UND boolean
8	^	Bitweises XOR
8	^	Logisches XOR
9		Bitweises ODER
9		Logisches ODER
10	&&	UND mit Kurzschluss
11		ODER mit Kurzschluss
12	?:	Bedingungsoperator
13	=	Zuweisung
14	*=, /=, %=, +=, -=	
14	<<=, >>=, >>>=	
14	&=, ^=, =	

Scanner

```
import java.util.Scanner;
Scanner scanner = new Scanner(System.in);
scanner.nextLine(); // Beispiel zum einlesen der naechsten Zeile
scanner.nextInt(); // Beispiel zum einlesen des naechstes Integers
```

Variablen/Identifier

Bestehen aus: Unicode Buchstaben, Ziffern, Währungssymbolen, Verbindungszeichen (_,), sind theoretisch beliebig lang.

Einschränkung: Keine Ziffern am Anfang, keine Leerzeichen

Deklaration Syntax

type variableName = value;

```
int myNum = 5;
float myFloatNum = 5.99f;
double myDouble = 5.12;
char myLetter = 'D';
boolean myBool = true;
String myText = "Hello";
final int MINUTES_PER_HOUR = 60; //KONSTANTE
```

Bezeichner

Legale Bezeichner	Illegale Bezeichner
Test	2cool4you
__DAS_IST_S000_COOL__	nein danke
bóoléáñ	yolo!
DAGOBERT\$\$\$	private
\$€	

Datentypen

Primitive Datentypen

byte, short, int, long, float, double, boolean, char

Nicht primitive Datentypen

Zum Beispiel: String, Arrays aber auch Instanzen von selbst erstellten

Klassen wie Auto auto = new Auto();

Wissenschaftliche Notation von Fließkommazahlen

```
float f1 = 35e3f;
double d1 = 12E4d;
```

Type Casting

Widening Casting (automatisch)

byte -> short -> char -> int -> long -> float -> double

Klein in groß geht immer und passiert automatisch!

Narrowing Casting (manuell!)

double -> float -> long -> int -> char -> short -> byte

Groß in klein geht aber nur mit Verlust (wie bei double in int Verlust der Nachkommastellen) und muss zum Beispiel mit (int) explizit gecastet werden! Also: double x = 9.7; int y = (int)x //Ausgabe: 9

Strings

Methoden

```
String str = "Hallo";
str.length(); // 5 (Gibt Laenge des Strings zurueck)
str.charAt(1); // 'a' (Gibt Zeichen an bestimmter Position zurueck)
str.toUpperCase(); // "HALLO" (Wandelt alles in Grossbuchstaben um)
str.toLowerCase(); // "hallo" (Wandelt alles in Kleinbuchstaben um)
```

Strings miteinander vergleichen

Bei Strings kann man nicht die Vergleichsoperatoren benutzen. Um Strings miteinander zu vergleichen benutzt man die Methode equals()

```
String a = "Hallo Welt";
String b = "Welt Hallo";
System.out.print(a.equals(b)); // Ausgabe: false
```

Verkettung von Strings

Strings können mit der Addition miteinander verkettet werden.

```
String hello = "Hello";
String world = "World";
String verkettung = hello+world;//ergibt "HelloWorld"
```

String Format

Mit String.format() und System.out.printf() können Strings mithilfe von Platzhaltern formatiert werden. Bei Floats/Doubles kann man Nachkommastellen mit angeben indem man statt %f dann %.2f schreibt. Im Beispiel steht die Zwei für die Menge an Nachkommastellen.

String = %s	int = %d	double/float = %f	Zeilenumbruch = %n
-------------	----------	-------------------	--------------------

```
String name = "Anna"; int alter = 25;
double kontostand = 1234.56789;
System.out.printf("Name: %s | Alter: %d | Kontostand: %.2f Euro%n",
    name, alter, kontostand);
//Ausgabe: Name: Anna | Alter: 25 | Kontostand: 1234.57 Euro
```

Escape Sequenzen in Strings

\n Neue Zeile, \t Tab, \b Backspace, \\ Backslash

Beispiel: "Mein String bekommt hier \n eine neue Zeile"

Java Math Klasse

Nützliche Methoden

Math.max(x,y), Math.min(x,y), Math.sqrt(x), Math.abs(x), Math.pow(x, y), Math.random()

If-Else

Short hand If-Else

variable=(condition)? expressionTrue : expressionFalse;

Switch

Anstatt vieler If-Else Anweisungen kann man eventuell auch einen Switch nehmen. Die Switch expression wird einmal ausgewertet. Das Ergebnis wird mit jedem einzelnen Wert aus den Cases verglichen (hier x und y). Wenn es eine Übereinstimmung gibt dann wird der Case ausgeführt. Das break; Statement stoppt den Switch direkt.

```
switch(expression) {
    case x:
        // code block fuer case x
        break;
    case y:
        // code block fuer case y
        break;
    default:
        // code block wenn kein case zutrifft
}
```

Schleifen

While Schleife

Wiederholt den Code in der While Schleife so lange wie die angegebene Condition true ist. Wenn die Condition von Anfang an false ist wird der Code nicht ausgeführt!

```
while (condition) {
    //code der wiederholt ausgefuert wird wenn condition true
}
```

Do-While Schleife

Die Do-While ist eine Variante der While Schleife, bei der der Code einmal ausgeführt wird BEVOR die Bedingung geprüft wird. Das heißt auch wenn die Bedingung falsch ist wird es auf jeden Fall einmal ausgeführt!

```
do {
    //code der mindestens einmal ausgefuert wird bevor condition geprueft
    wird und dann weiterhin ausgefuehrt wird solange die condiiton
    true ist
}
while (condition);
```

For Schleife

Wenn man ganz genau weiß, wie oft man einen Code ausführen möchte benutzt man die For statt einer While Schleife. Zum Beispiel wenn man über einen kompletten String iterieren möchte weiß man genau wie oft. Und zwar String.length() mal. Statement 1 wird ein einziges mal ausgeführt BEVOR die For Schleife beginnt. Statement 2 ist die Condition wie lange die For ausgeführt werden soll. Statement 3 wird nach jedem Durchgang des Codeblocks der For Schleife einmal ausgeführt.

```
for (statement 1; statement 2; statement 3) {
    // code der wiederholt ausgefuehrt werden soll
}
```

```
for(int i = 0; i < 10; i++){
    //code wird 10 mal ausgefuehrt, weil von i=0 bis i=9
}
```

Verschachtelte Schleifen

Schleifen können auch ineinander verschachtelt werden. Die innere Schleife wird einmal komplett für jeden Durchgang der äußeren Schleife ausgeführt. Als Beispiel könnte man sich eine Tabelle vorstellen. Die äußere Schleife geht die Zeilen durch und die innere Schleife geht die Spalten durch.

```
for (int i = 1; i <= 4; i++) { //Auessere Schleife
    for (int j = 1; j <= 3; j++) { //Innere Schleife
        System.out.print(i * j + " "); // wird 12 mal ausgefuehrt (4 * 3)
    }
    System.out.println(); //wird 3 mal ausgefuehrt
}/* Ausgabe ist:
1 2 3
2 4 6
3 6 9
4 8 12 */
```

For-Each Schleife

Es gibt auch eine For-Each Schleife. Diese wird eigentlich ausschließlich dafür benutzt, um durch die Elemente in einem Array zu iterieren. Diese Schleife ist etwas einfacher und lesbarer, da man auch keinen Counter brauch um zu itereieren.

```
for (type variableName : arrayName) {
    // code
}

String[] cars = {"Volvo", "BMW", "Ford", "Mazda"};
for (String car : cars) {
    System.out.println(car);
} /* Ausgabe:
Volvo
BMW
Ford
Mazda */
```

Break und Continue

Break

Das **break**; Statement kennen wir schon aus dem Switch. Das **break**; Statement kann auch benutzt werden um direkt eine Schleife abzubrechen. Dabei wird die komplette Schleife abgebrochen und nicht mehr weiter durchgegangen, egal wie viele Durchgänge noch gekommen wären.

Continue

Das **continue**; Statement wird benutzt um die aktuelle Iteration abzubrechen und direkt mit der nächsten Iteration weiter zu machen. Hier wird also kein kompletter Abbruch der Schleife durchgeführt wie beim Break sondern nur ein Abbruch der aktuellen Iteration und es wird zur nächsten Iteration gesprungen.

Arrays

Arrays werden benutzt um mehrere Werte des gleichen Datentyps zu speichern. Arrays können aus allen möglichen Datentypen gebaut werden. Sowohl Primitive (**int**[], **short**[], **double**[] usw...) als auch nicht Primitive(**String**[], **Auto**[]). Ein Array beginnt IMMER mit dem index 0. Ein 10 Elementiges Array hat also 10 Elemente und die Indizes der Elemente gehen von 0 bis 9

Deklaration

```
//Deklaration Beispiele
String[] cars;
int[] zahlen;
//Deklaration mit direkter Zuweisung von Werten
String[] cars = {"Volvo", "BMW", "Ford", "Mazda"};
int[] zahlen = {10, 20, 30, 40};
//Deklaration mit bestimmter groesse ohne Zuweisung
String[] cars = new String[4];
int[] zahlen = new int[5];
```

Auf Elemente eines Arrays zugreifen

```
String[] cars = {"Volvo", "BMW", "Ford", "Mazda"};
System.out.println(cars[0]); // auf das nullte Element zugreifen also
Ausgabe: Volvo
```

Ein bestimmtes Element eines Arrays verändern

```
cars[0] = "Opel"; //Element mit Index 0 wird auf "Opel" gesetzt
```

Iterieren eines Arrays

```
int[] numbers = {10, 20, 30, 40};
//loop mit normaler for
for (int i = 0; i < numbers.length; i++) {
    System.out.println(numbers[i]);
}

String[] cars = {"Volvo", "BMW", "Ford", "Mazda"};
//loop mit for-each
for (String car : cars) {
    System.out.println(car);
}
```

Array Länge

Um die größe eines Arrays rauszufinden benutzt man das length Attribut (**cars.length**) des Arrays. Vorsicht auch hier gilt bei einem 10 Elementigen Array ist length = 10 aber Indizes von 0 bis 9!

Multidimensionales Array

Das Array hat in beiden Deklarationsfällen 2 Zeilen und 3 Spalten.

```
//Deklaration mit gegebenen Werten
int[][] meineZahlen = { {1, 4, 2}, {3, 6, 8} };
//Deklaration ohne gegebene Werte
int[][] meineZahlen = new int[2][3]; //2 Zeilen 3 Spalten
//Auf bestimmte Elemente zugreifen
meineZahlen[1][2] //Element aus Zeile 1 Spalte 2
//Ein bestimmtes Element aendern
meineZahlen[1][0] = 9; //Element aus Zeile 1 Spalte 0 aendern
//laenge der Reihen
meineZahlen.length
//laenge der Spalten einer bestimmten Reihe
meineZahlen[1].length
```

Iterieren eines Multidimensionalen Arrays

```
int[][] meineZahlen = { {1, 4, 2}, {3, 6, 8, 5, 2} };

for (int row = 0; row < meineZahlen.length; row++) {
    for (int col = 0; col < meineZahlen[row].length; col++) {
        System.out.println("meineZahlen[" + row + "][" + col + "] = " +
            meineZahlen[row][col]);
    }
} /* Ausgabe:
meineZahlen[0][0] = 1
meineZahlen[0][1] = 4
meineZahlen[0][2] = 2
meineZahlen[1][0] = 3
meineZahlen[1][1] = 6
meineZahlen[1][2] = 8
meineZahlen[1][3] = 5
meineZahlen[1][4] = 2 */
```

Zweierpotenzen

2 ¹⁰	2 ⁹	2 ⁸	2 ⁷	2 ⁶	2 ⁵	2 ⁴	2 ³	2 ²	2 ¹	2 ⁰
1024	512	256	128	64	32	16	8	4	2	1

Primitive Datentypen

Typ	byte	short	char	int	long	float	double
Bit	8	16	16	32	64	32	64

Methoden

public, private, protected ist Sichtbarkeit der Methode. **void, int, String** usw. sind die Rückgabewerte. Bei **void** wird nichts zurückgegeben braucht

also kein **return**. Parameter werden in die Klammer geschrieben wie bei **meineMethode(int x)**. Mehrere Parameter mit Komma getrennt.

```
public static void meineMethode() {
    //Methode ohne Paramter und ohne Rueckgabewert
}

public int meineMethode(int x) {
    //Methode mit Parametern und Rueckgabewert
    return 5;
}
```

Rekursion

Methoden die sich selbst aufrufen. Kann man mit Schleifen vergleichen, ist aber eine ganz andere Technik.

```
public static int sum(int k) {
    if (k > 0) {
        return k + sum(k - 1);
    } else {
        return 0;
    }
}
```

Syntax vs Semantik

Syntax beschreibt die Regeln bzw. die "Grammatik". Also mit anderen Worten wie genau muss etwas geschrieben werden, damit es syntaktisch korrekt ist. (Zeilen werden mit Semikolon beendet, wie genau ist eine For-Schleife aufgebaut etc.).

Semantik ist der Sinn innerhalb des Codes. Dabei geht es um Bedeutung des Codes.

Call by Value/Reference

Call by Value

Call bei Value passiert bei primitiven Datentypen. Dabei wird der Methode nur eine Kopie weitergegeben. Das heißt, der Wert der übergeben wurde ist unabhängig von dem Wert mit dem die Methode arbeitet. Wird der Wert innerhalb der Methode bearbeitet hat das keinen Einfluss außerhalb der Methode.

Call by Reference

Call by Reference passiert bei nicht Primitiven Datentypen. Zu diesen Datentypen gibt es nur eine Referenz, die auf eine Speicheradresse im Speicher zeigt. Wenn man das an eine Methode weitergibt, dann wird der Methode die Referenz übergeben und nicht wie bei Call by Value eine Kopie erzeugt. Dadurch, dass die Methode nun die genaue Referenz auf das Objekt im Speichert hat kann die Methode dieses Objekt verändern und das ist dann auch außerhalb der Methode spürbar.

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum. Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.